R Programming Language

Dr. Anupam Ghosh

Overview

- Communication with R
- History
- Introduction to R
- R with Statistics and Data Analytics
- Who uses R today?
- Installing and Running of R
- Packages of R
- Data Type and OOPs concept
- R workspace, Operators, Scripting
- Visualization
- Regression

Communication with R

- The R/S language has become the most common language for communication in the fields of Statistics and Data Analysis
- The R statistical programming language is a free open source package based on the S language developed by Bell Labs
- Books are being written now with R presented directly placed within the text
- Excellent for research
- Excellent for teaching
- Excellent for Data Analytics

History

- S: an interactive environment for data analysis developed at Bell Laboratories since 1976
  - 1988 - S2: RA Becker, JM Chambers, A Wilks
  - 1992 - S3: JM Chambers, TJ Hastie
  - 1998 - S4: JM Chambers
- Exclusively licensed by *AT&T/Lucent* to *Insightful Corporation*, Seattle WA. Product name: "S-plus".
- Implementation languages C, Fortran
  http://cm.bell-labs.com/cm/ms/departments/sia/S/history.html
- R: initially written by Ross Ihaka and Robert Gentleman at Dep. of Statistics of U of Auckland, New Zealand during 1990s.
- Since 1997: international "R-core" team of ca. 15 people with access to common CVS archive.

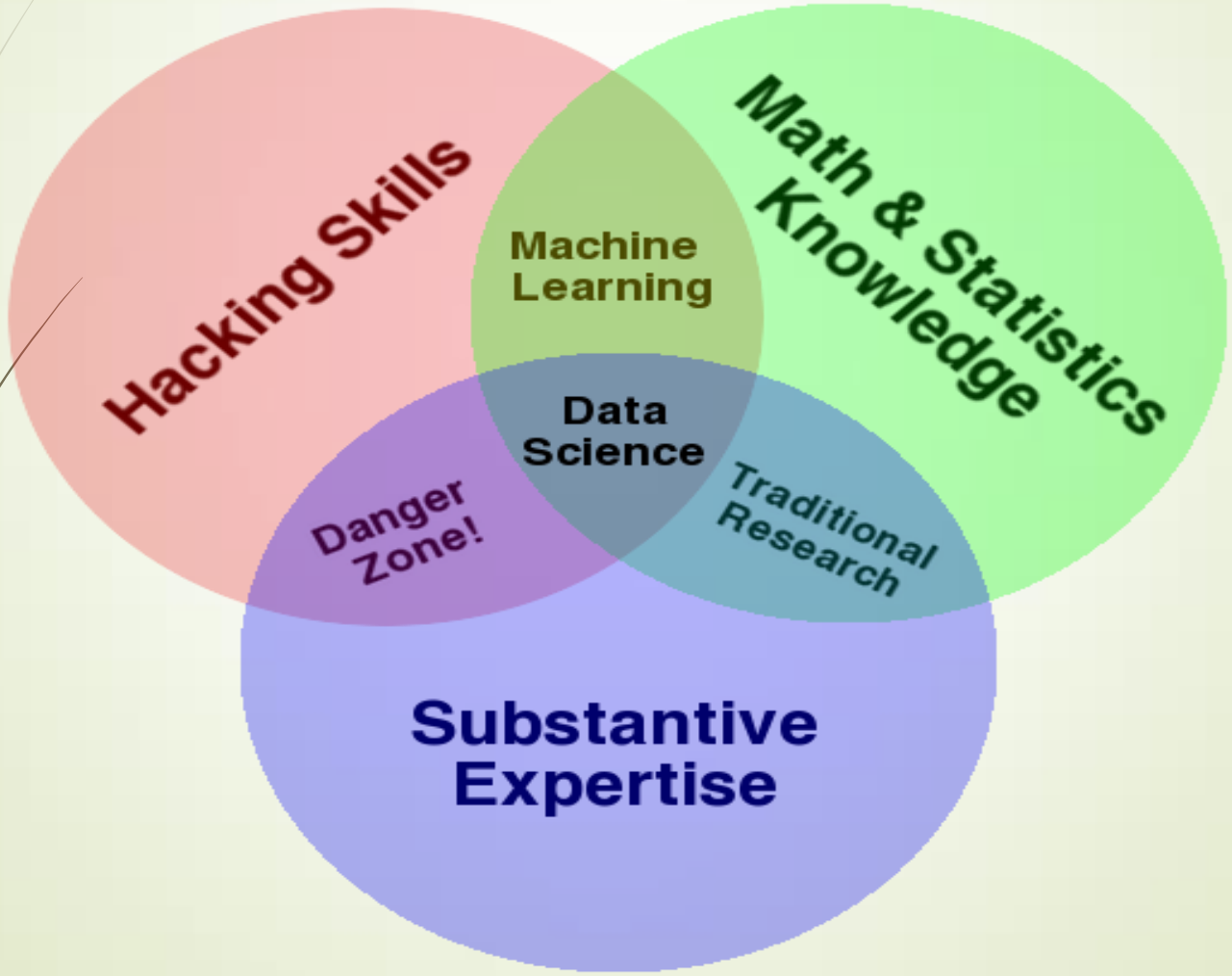
Introduction

"GNU S" — A language and environment for data manipulation, calculation and graphical display

- R is similar to the award-winning S system, which was developed at Bell Laboratories by John Chambers et al.
- A suite of operators for calculations on arrays (matrices), a large coherent, integrated collection of intermediate tools for interactive data analysis
- A well developed programming language which includes conditionals, loops, user defined recursive functions and input and output facilities.

- The core of R is an interpreted computer language
  - It allows branching and looping as well as modular programming using functions.
  - Most of the user-visible functions in R are written in R, calling upon a smaller set of internal primitives.
  - It is possible for the user to interface to procedures written in C, C++ or FORTRAN languages for efficiency, and also to write additional primitives

Why R in Data science?



R and Statistics

- Packaging: a crucial infrastructure to efficiently produce, load and keep consistent software libraries from (many) different sources / authors
- Statistics: most packages deal with statistics and data analysis
- State of the art: many statistical researchers provide their methods as R packages

Data Analysis and Presentation

- The R distribution contains functionality for large number of statistical procedures
  - linear and generalized linear models
  - nonlinear regression models
  - time series analysis
  - classical parametric and nonparametric tests
  - clustering
  - Smoothing
  - Machine/Ensemble Learning Packages
- R also has a large set of functions which provide a flexible graphical environment for creating various kinds of data presentations.

http://www.revolutionanalytics.com/what-is-open-source-r/companies-using-r.php

Sector
Asia Capital Reinsurance Chooses Revolution DeployR to Analyze Complex Data and Generate Business Insights

ANZ Bank and R
ANZ, the fourth largest bank in Australia, using R for credit risk analysis

Savng the World with R
Benetech is a non-profit organization contracted by the likes of Amnesty International and Human Rights Watch to answer thorny geopolitical questions through the use of data and science.

Social Search Awesomeness
Jesse Bridgewater works on "social search awesomeness" for the Bing search engine, and is setting up his dev environment with the necessary tools including python, vim, and R.

Complex Data Sets in Genomic Diagnostics Require Multiple Analytic Methods
Revolution Helps Accelerate Process, Reducing Project Time

Facebook and R

- Analysis of Facebook Status Updates
- Facebook's Social Network Graph
- How Google and Facebook are using R
- Predicting Colleague Interactions with R

Google and R

- Google uses R to Predict Economic Activity
- More R at Google
- Google using R to Analyze Effectiveness of TV ads

How to download R?

Google it using R or CRAN

(Comprehensive R Archive Network)

http://www.r-project.org

Select a CRAN mirror

Click 'windows'

Click 'base'

Click 'Download R 3.2.3 for Windows'(67 megabytes, 32/64 bit)

Save 'R-3.2.3-win.exe'

R Studio: Platform

https://www.rstudio.com/products/RStudio/

Running R

RGui (32-bit) - [R Console]

File Edit View Misc Packages Windows Help

```
R version 3.2.3 (2015-12-10) -- "Wooden Christmas-Tree"
Copyright (C) 2015 The R Foundation for Statistical Computing
Platform: i386-w64-mingw32/i386 (32-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[Previously saved workspace restored]

> x=3
> y<-5
> z<-x+y
> z
[1] 8
>
```

R package site

The Comprehensive R Archive

cran.r-project.org

The Comprehensive R Archive Network

CRAN
Mirrors
What's new?
Task Views
Search

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Other

Documentation
Manuals
FAQs
Contributed

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- Download R for Linux
- Download R for MacOS X
- Download R for Windows

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2012-03-30, Easter Beagle): R-2.15.0.tar.gz, read what's new in the latest version.
- Sources of R alpha and beta releases (daily snapshots, created only in time periods before a planned release).
- Daily snapshots of current patched and development versions are available here. Please read about new features and bug fixes before filing corresponding feature requests or bug reports.
- Source code of older versions of R is available here.
- Contributed extension packages

Questions About R

Click

For time series analysis package

TSA: Time Series Analysis

Contains R functions and datasets detailed in the book "Time Series Analysis with Applications in R (second edition)" by Jonathan Cryer and Kung-Sik Chan

Version:	1.01
Depends:	R (≥ 2.5.1), leaps, locfit, mgcv, tseries
Published:	2012-11-13
Author:	Kung-Sik Chan, Brian Ripley
Maintainer:	Kung-Sik Chan <kungsik.chan at gmail.com>
License:	GPL-2 | GPL-3 [expanded from: GPL (≥ 2)]
URL:	http://www.stat.uiowa.edu/~kchan/TSA.htm
NeedsCompilation:	no
In views:	Distributions, TimeSeries
CRAN checks:	TSA results

Downloads:

Reference manual:	TSA.pdf
Package source:	TSA_1.01.tar.gz
MacOS X binary:	TSA_1.01.tgz
Windows binary:	TSA_1.01.zip

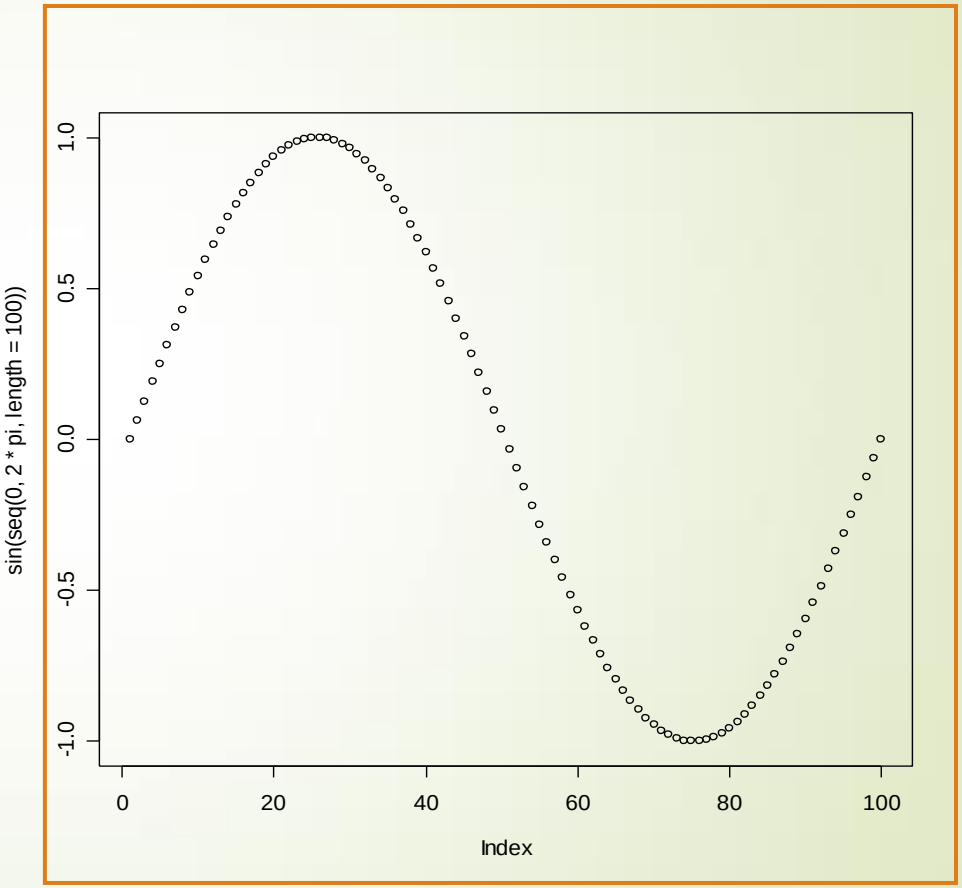
Then, click on this blue link "TSA_1.01.zip" and save into your computer.

R scripting

- To print the object just enter the name of the object. There are some restrictions when giving an object a name:
  - Object names cannot contain `strange' symbols like !, +, -, #.
  - A dot (.) and an underscore (_) are allowed, also a name starting with a dot
  - Object names can contain a number but cannot start with a number.
  - R is case sensitive, X and x are two different objects, as well as Foo and foo

R as a calculator


```
> log2(32)
[1] 5
> sqrt(2)
[1] 1.414214
> seq(0, 5, length=6)
[1] 0 1 2 3 4 5
> plot(sin(seq(0, 2*pi, length=100)))
```



R code

```
> 2+2
[1] 4
> 2+2^2
[1] 6
> (2+2)^2
[1] 16
```

```
> sqrt(2)
[1] 1.414214
> log(2)
[1] 0.6931472
> x = 5
> y = 10
> z <- x+y
> z
[1] 15
```

R Code

```
> seq(1,5, by=.5)
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
> v1 = c(6,5,4,3,2,1)
> v1
[1] 6 5 4 3 2 1
> v2 = c(10,9,8,7,6,5)
> v3 = v1 + v2
> v3
[1] 16 14 12 10  8  6
> max(v3);min(v3)
[1] 16; [1] 6
> length(v3)
[1] 6
> mean(v3)
[1] 11
> sd(v3)
[1] 3.741657
```

Data Type

primitive (or: atomic) data types in R are:

- **numeric (integer, double, complex)**
- **character**
- **logical**
- **function**

out of these, vectors, arrays, lists can be built as Python

Variables


```
> a = 49
> sqrt(a)
[1] 7

> a = "The dog ate my homework"
> sub("dog","cat",a)
[1] "The cat ate my homework“

> a = (1+1==3)
> a
[1] FALSE
```

numeric

character string

logical

Vectors, Matrices and Arrays

- vector: an ordered collection of data of the same type

```
> a = c(1,2,3)
> a*2
[1] 2 4 6
```

- Example: the mean spot intensities of all 15488 spots on a chip

- matrix: a rectangular table of data of the same type
- example: the expression values for 10000 genes for 30 tissue biopsies: a matrix with 10000 rows and 30 columns.

```
> m<- matrix (1:6,nrow=2, ncol=3)
```

- dim(m)

- array: 3-,4-,..dimensional matrix

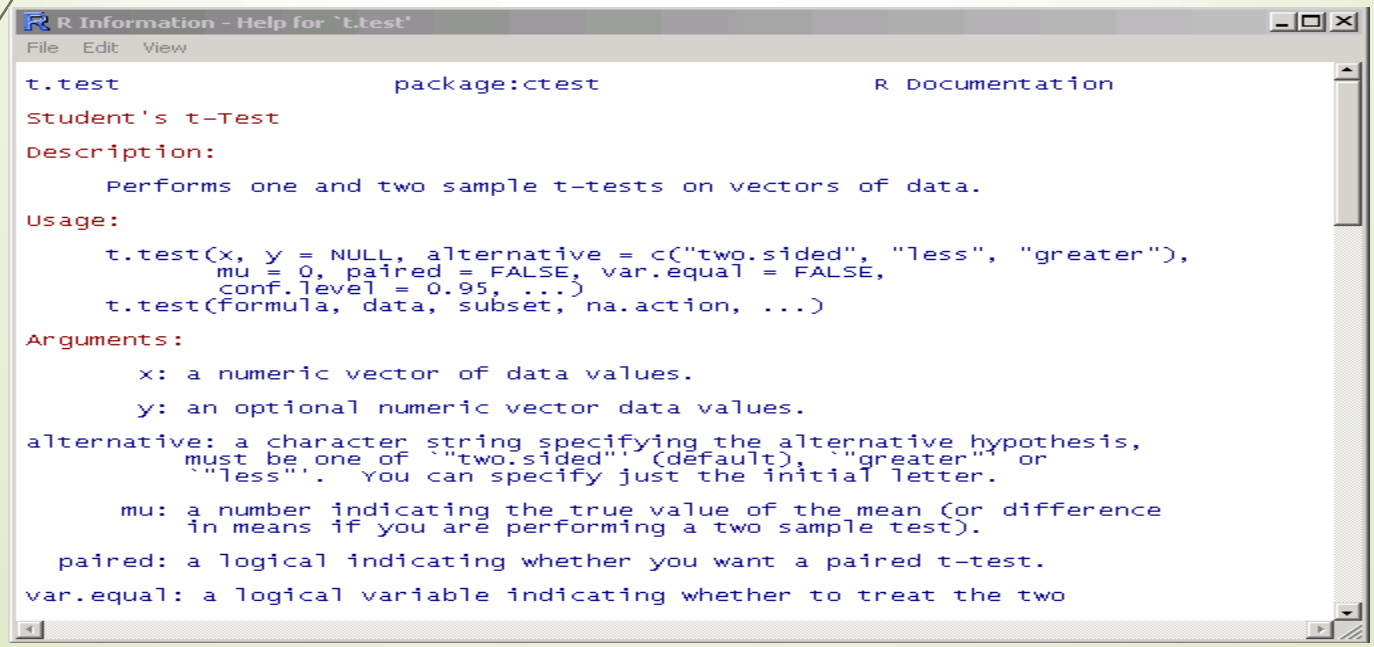
- example: the red and green foreground and background values for 20000 spots on 120 chips: a 4 x 20000 x 120 (3D) array

Data Frame and list

- Data Frames:
  - Group a collection of related vectors
  - Load from a text file using read.table()
  - Parameters header, sep, and na.strings control
  - useful options read.csv() and read.delim() have useful defaults for comma or tab delimited files
- List:
  - Collections of related variables
  - Similar to records in C
  - Created with list function: point <- list(x = 1, y = 1)
  - Access to components follows similar rules as for data frames, the following all retrieve x:
  - point$x; point[“x”]; point[1]; point[-2]

Getting help

- help.start() #Take a look
- help.search("histogram") will search for topics that include the word histogram
- >? t.test / >help(t.test)



```
R Information - Help for `t.test'
File  Edit  View

t.test                    package:ctest                    R Documentation

Student's t-Test

Description:

     Performs one and two sample t-tests on vectors of data.

Usage:

     t.test(x, y = NULL, alternative = c("two.sided", "less", "greater"),
            mu = 0, paired = FALSE, var.equal = FALSE,
            conf.level = 0.95, ...)
     t.test(formula, data, subset, na.action, ...)

Arguments:

       x: a numeric vector of data values.

       y: an optional numeric vector data values.

alternative: a character string specifying the alternative hypothesis,
          must be one of `"two.sided"' (default), `"greater"' or
          `"less"'.  You can specify just the initial letter.

      mu: a number indicating the true value of the mean (or difference
          in means if you are performing a two sample test).

  paired: a logical indicating whether you want a paired t-test.

var.equal: a logical variable indicating whether to treat the two
```

Arithmetic and Logical Operators

Operator	Description
+	addition
-	subtraction
*	multiplication
/	division
^ or **	exponentiation
x %% y	modulus (x mod y) 5%%2 is 1
x %/% y	integer division 5%/%2 is 2

Operator	Description
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
==	exactly equal to
!=	not equal to
!x	Not x
x | y	x OR y
x & y	x AND y
isTRUE(x)	test if x is TRUE

Control Structures

- **R** has the standard control structures you would expect. **expr** can be multiple (compound) statements by enclosing them in braces { } like C. It is more efficient to use built-in functions rather than control structures whenever possible
  - **if-else**
    - if (*cond*) *expr*
      if (*cond*) *expr1* else *expr2*
  - **for**
    - for (*var* in *seq*) *expr*
  - **while**
    - while (*cond*) *expr*
  - **switch**
    - switch(*expr*, ...)
  - **ifelse**
    - ifelse(*test*,*yes*,*no*)

  Repeat, break, next……

Numeric & Character Functions

Function	Description
abs(*x*)	absolute value
sqrt(*x*)	square root
ceiling(*x*)	ceiling(3.475) is 4
floor(*x*)	floor(3.475) is 3
trunc(*x*)	trunc(5.99) is 5
round(*x*, digits=*n*)	round(3.475, digits=2) is 3.48
signif(*x*, digits=*n*)	signif(3.475, digits=2) is 3.5
cos(*x*), sin(*x*), tan(*x*)	also acos(*x*), cosh(*x*), acosh(*x*), etc.
log(*x*)	natural logarithm
log10(*x*)	common logarithm
exp(*x*)	e^*x*

Function	Description
substr(*x*, start=*n1*, stop=*n2*)	Extract or replace substrings in a character vector. x <- "abcdef" substr(x, 2, 4) is "bcd" substr(x, 2, 4) <- "22222" is "a222ef"
grep(*pattern*, *x* , ignore.case=FALSE, fixed=FALSE)	Search for *pattern* in *x*. If fixed =FALSE then *pattern* is a regular expression. If fixed=TRUE then *pattern* is a text string. Returns matching indices. grep("A", c("b","A","c"), fixed=TRUE) returns 2
sub(*pattern*, *replacement*, *x*, ignore.case =FALSE, fixed=FALSE)	Find *pattern* in *x* and replace with *replacement* text. If fixed=FALSE then *pattern* is a regular expression. If fixed = T then *pattern* is a text string. sub("\\s",".","Hello There") returns "Hello.There"
strsplit(*x*, *split*)	Split the elements of character vector *x* at *split*. strsplit("abc", "") returns 3 element vector "a","b","c"
paste(..., sep="")	Concatenate strings after using *sep* string to seperate them. paste("x",1:3,sep="") returns c("x1","x2" "x3") paste("x",1:3,sep="M") returns c("xM1","xM2" "xM3") paste("Today is", date())
toupper(*x*)	Uppercase
tolower(*x*)	Lowercase

User defined function in R

- Easy to create your own functions in R
  - As tasks become complex, it is a good idea to organize code into functions that perform defined tasks
  - In R, it is good practice to give default values to function arguments
- Function definition:
  - name **<- function(**arg1, arg2, …**)** expression
  - Arguments can be assigned default values:

    arg_name **=** expression
- Return value is the last evaluated expression or can be set explicitly with return()

User defined function in r: Example

```
square <- function(x = 10) x * x
> square()
[1] 100
> square(2)
[1] 4
> intsum <- function(from=1, to=10){
sum <- 0
for (i in from : to)
sum <- sum + i
Sum }
> intsum(3)                    # Evaluates sum from 3 to 10 …
[1] 52
> intsum(to = 3)               # Evaluates sum from 1 to 3 …
[1] 6
```

lapply, sapply, apply

- When the same or similar tasks need to be performed multiple times for all elements of a list or for all columns of an array.
  - May be easier and faster than “for” loops
- lapply(li, *function*)
  - To each element of the list li, the function is applied
  - The result is a list whose elements are the individual *function* results.

```
> li = list("klaus","martin","georg")
> lapply(li, toupper)
> [[1]]
> [1] "KLAUS"
> [[2]]
> [1] "MARTIN"
> [[3]]
> [1] "GEORG"
```

lapply, sapply, apply(contd..)

- sapply(li, fct)
- Like lapply, but tries to simplify the result, by converting it into a vector or array of appropriate size
- > li = list("klaus","martin","georg")

```
> sapply(li, toupper)
[1] "KLAUS"  "MARTIN" "GEORG"
> fct = function(x) { return(c(x, x*x, x*x*x)) }
> sapply(1:5, fct)
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    2    3    4    5
[2,]    1    4    9   16   25
[3,]    1    8   27   64  125
```

lapply, sapply, apply(contd..)

- apply(arr, margin, fct)
- Apply the function fct along some dimensions of the array arr, according to margin, and return a vector or array of the appropriate size.
- > x
- [,1] [,2] [,3]
- [1,] 5 7 0
- [2,] 7 9 8
- [3,] 4 6 7
- [4,] 6 3 5
- > apply(x, 1, sum)
- [1] 12 24 17 14
- > apply(x, 2, sum)
- [1] 22 25 20

Stat/Prob Functions

Function	Description

dnorm(*x*)	normal density function (by default m=0 sd=1)
	# plot standard normal curve
	x <- pretty(c(-3,3), 30)
	y <- dnorm(x)
	plot(x, y, type='l', xlab="Normal Deviate", ylab="Density", yaxs="i")

pnorm(*q*)	cumulative normal probability for q (area under the normal curve to the right of q) pnorm(1.96) is 0.975
rnorm(*n*, m=0,sd=1)	n random normal deviates with mean m and standard deviation sd. #50 random normal variates with mean=50, sd=10 x <- rnorm(50, m=50, sd=10)

dbinom(*x*, *size*, *prob*)
pbinom(*q*, *size*, *prob*)
qbinom(*p*, *size*, *prob*)
rbinom(*n*, *size*, *prob*)
	binomial distribution where size is the sample size
	and prob is the probability of a heads (pi)
	# prob of 0 to 5 heads of fair coin out of 10 flips
	dbinom(0:5, 10, .5)
	# prob of 5 or less heads of fair coin out of 10 flips
	pbinom(5, 10, .5)

mean(*x*, trim=0, **na.rm=FALSE)**	mean of object x # trimmed mean, removing any missing values and # 5 percent of highest and lowest scores mx <- mean(x,trim=.05,na.rm=TRUE)
sd(*x*)	standard deviation of object(x). also look at var(x) for variance and mad(x) for median absolute deviation.

Other Useful Functions

Function	Description
seq(*from* **,** *to***,** *by***)**	generate a sequence indices <- seq(1,10,2) #indices is c(1, 3, 5, 7, 9)
rep(*x***,** *ntimes***)**	repeat *x n* times y <- rep(1:3, 2) # y is c(1, 2, 3, 1, 2, 3)
cut(*x***,** *n***)**	divide continuous variable in factor with *n* levels y <- cut(x, 5)

Sorting

- To sort a dataframe in R, use the **order( )** function
  - By default, sorting is ASCENDING. Prepend the sorting variable by a minus sign to indicate DESCENDING order

```
# sorting examples using the mtcars dataset
data(mtcars)
newdata = mtcars[order(mtcars$mpg),]      #sort by mpg

newdata <- mtcars[order(mtcars$mpg, mtcars$cyl),]
                                          #sort by mpg and cyl
newdata <- mtcars[order(mtcars$mpg, -mtcars$cyl),]
                                          #sort by mpg (ascending) and cyl (descending)
```

Merging

- To merge two dataframes (datasets) horizontally, use the **merge()** function
  - In most cases, you join two dataframes by one or more common key variables (an inner join)

```
#merge two dataframes by ID
total <- merge(dataframeA, dataframeB, by="ID")
#merge two dataframes by ID and Country
total <- merge(dataframeA, dataframeB, by=c("ID","Country"))
```

Merging(Contd..)

- To join two dataframes (datasets) vertically, use the rbind() and cbind() function. The two dataframes must have the same variables, but they do not have to be in the same order.

total <- rbind(dataframeA, dataframeB)

```
> x<-1:3;  > y<-10:12
> rbind(x,y)
  [,1] [,2] [,3]
x    1    2    3
y   10   11   12
> cbind(x,y)
     x  y
[1,] 1 10
[2,] 2 11
[3,] 3 12
```

Storing data

- Every R object can be stored into and restored from a file with the commands “save” and “load”.
- This uses the XDR (external data representation) standard of Sun Microsystems and others, and is portable between MS-Windows, Unix, Mac.

```
> save(x, file=“x.Rdata”)
> load(“x.Rdata”)
```

Importing and exporting

- Help(read.csv)
- Data1<- read.csv(file.choose(), header=T)
- Data2<- read.table(file.choose(), header=T, sep=“,”)
- Data3<- read.delim(file.choose(), header=T)
- Data4<- read.table(file.choose(), header=T, sep=“\t”)
- Most programs (e.g. Excel), as well as humans, know how to deal with rectangular tables in the form of tab-delimited text files.

➢ x = read.delim(“filename.txt”)

also: read.table, read.csv,….

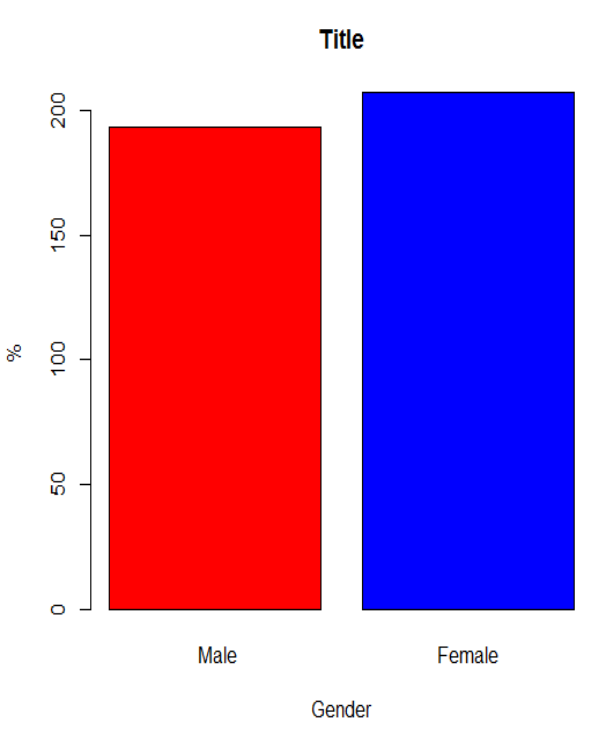
➢ write.table(x, file=“x.txt”, sep=“\t”)

also: write.csv, write.delim,….

Visualization

- Bar Graph:

```
> data2<-read.csv(file.choose(),header=T)
> head(data2)
> percent<-table(data2$Gender)
> percent
##  Male Female
## 193    207
```

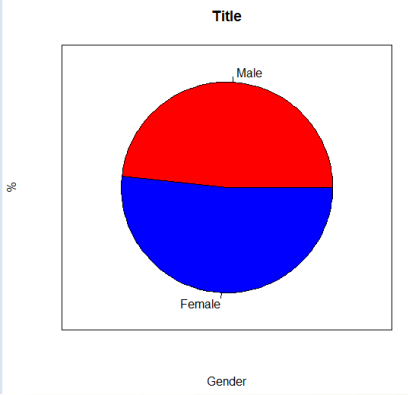



```
> barplot(percent,main="Title",xlab="Gender",ylab="%",col=c(2,4))
```

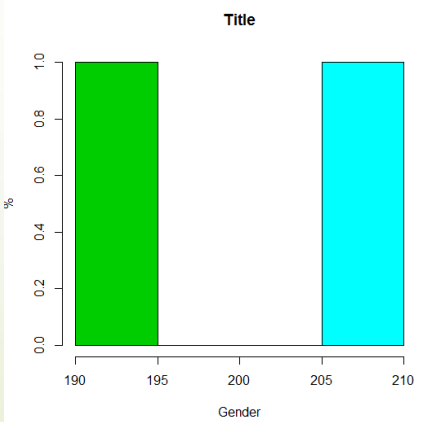
Visualization

- Pie Chart and Histogram:

```
> pie(percent,main="Title",xlab="Gender",ylab="%",col=c(2,4))
> box()
```




```
> hist(percent,main="Title",xlab="Gender",ylab="%",col=c(3,5))
```



Linear Regression

- Regression is to build a function of independent variables (also known as predictors) to predict a dependent variable (also called response)
- For example, banks assess the risk of home-loan applicants based on their age, income, expenses, occupation, number of dependents, total credit limit, etc.
- Linear regression is to predict response with a linear function of predictors as follows:

  $y = c_0 + c_1x_1 + c_2x_2 + \ldots + c_kx_k$; where x_1, x_2, …. ,x_k are predictors and y is the response to predict
- Linear regression with function lm()
- General linear regression with function glm()

Linear Regression on CPI data

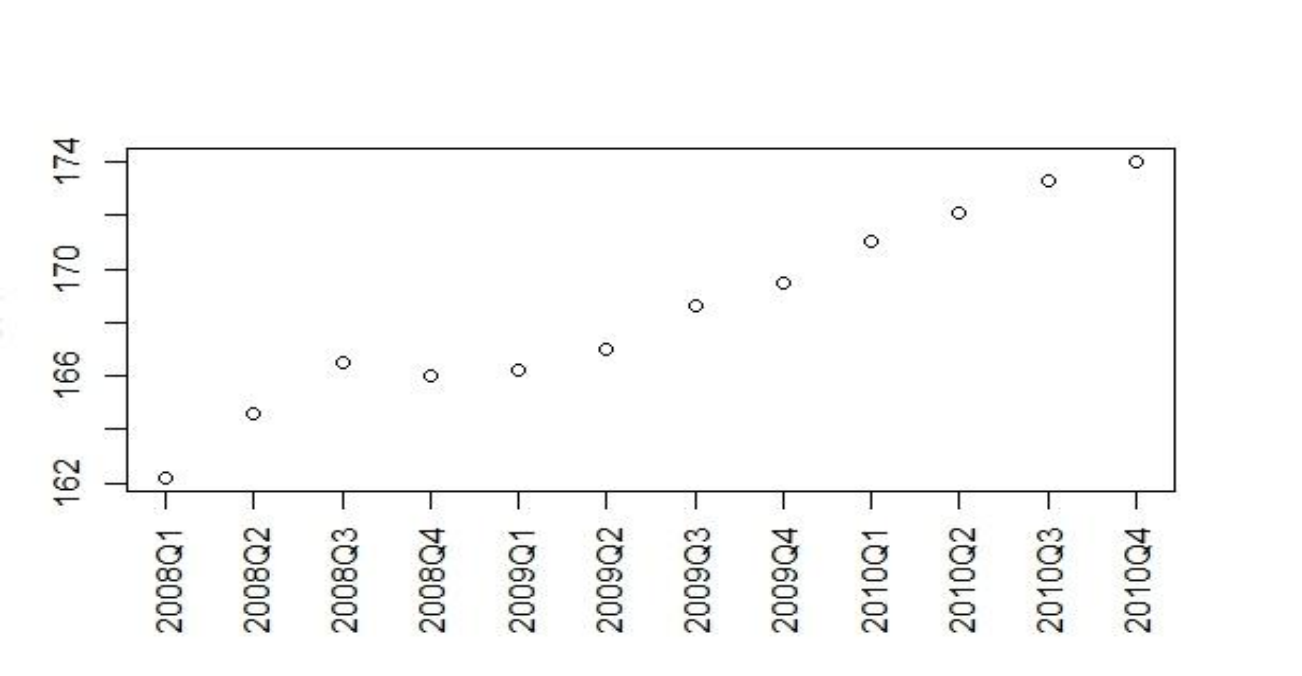
```
year <- rep(2008:2010, each = 4)

quarter <- rep(1:4, 3)

cpi <- c(162.2, 164.6, 166.5, 166, 166.2, 167, 168.6, 169.5, 171, 172.1, 173.3, 174)

plot(cpi, xaxt = "n", ylab = "CPI", xlab = "")  #draw x-axis, where 'las=3' makes text vertical

axis(1, labels = paste(year,  quarter,  sep  = "Q"), at = 1:12, las = 3)
```



Linear Regression

```
cor(year, cpi)          ## correlation between CPI and year / quarter
## [1] 0.9096
cor(quarter, cpi)
## [1] 0.3738
fit <- lm(cpi ~ year + quarter)  ## build a linear regression model with function lm()
fit
## Call:
## lm(formula = cpi ~ year + quarter)
##
## Coefficients:
## (Intercept)      year         quarter
## -7644.49         3.89           1.17
```

Linear Regression

With the above linear model, CPI is calculated as:

$$\mathbf{cpi = c_0 + c_1 * year + c_2 * quarter}$$

where c_0, c_1 and c_2 are coefficients from model fit.

What will the CPI be in 2011?

```
cpi2011 <- fit$coefficients[[1]] + fit$coefficients[[2]] * 2011 +
    fit$coefficients[[3]] * (1:4)
```

```
cpi2011
## [1] 174.4   175.6   176.8    177.9
```

An easier way is to use function predict()

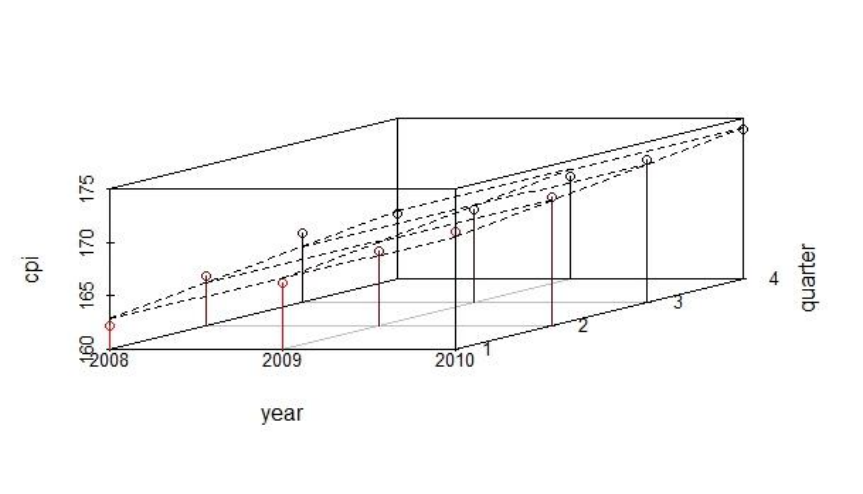
More details of the model can be obtained with the code below

```
attributes(fit)

## $names
## [1] "coefficients" "residuals" "effects"
## [4] "rank" "fitted.values" "assign"
## [7] "qr" "df.residual" "xlevels"
## [10] "call" "terms" "model"
##
## $class
## [1] "lm"
fit$coefficients
## (Intercept)      year        quarter
## -7644.488      3.888       1.167
```

Function residuals(): differences between observed values and predicted values

```
residuals(fit)            #differences between observed values and fitted values
## 1 2 3 4 5 6 ...
## -0.57917 0.65417 1.38750 -0.27917 -0.46667 -0.83333 -0.40...
## 8 9 10 11 12
## -0.66667 0.44583 0.37917 0.41250 -0.05417
summary(fit)              #shows summary format of fit
library(scatterplot3d)    #to show 3D view
s3d <- scatterplot3d(year, quarter, cpi, highlight.3d = T, type = "h", lab = c(2, 3))
                                        #lab: number of tickmarks on x-/y-axes
s3d$plane3d(fit)
```



Prediction of CPIs in 2011

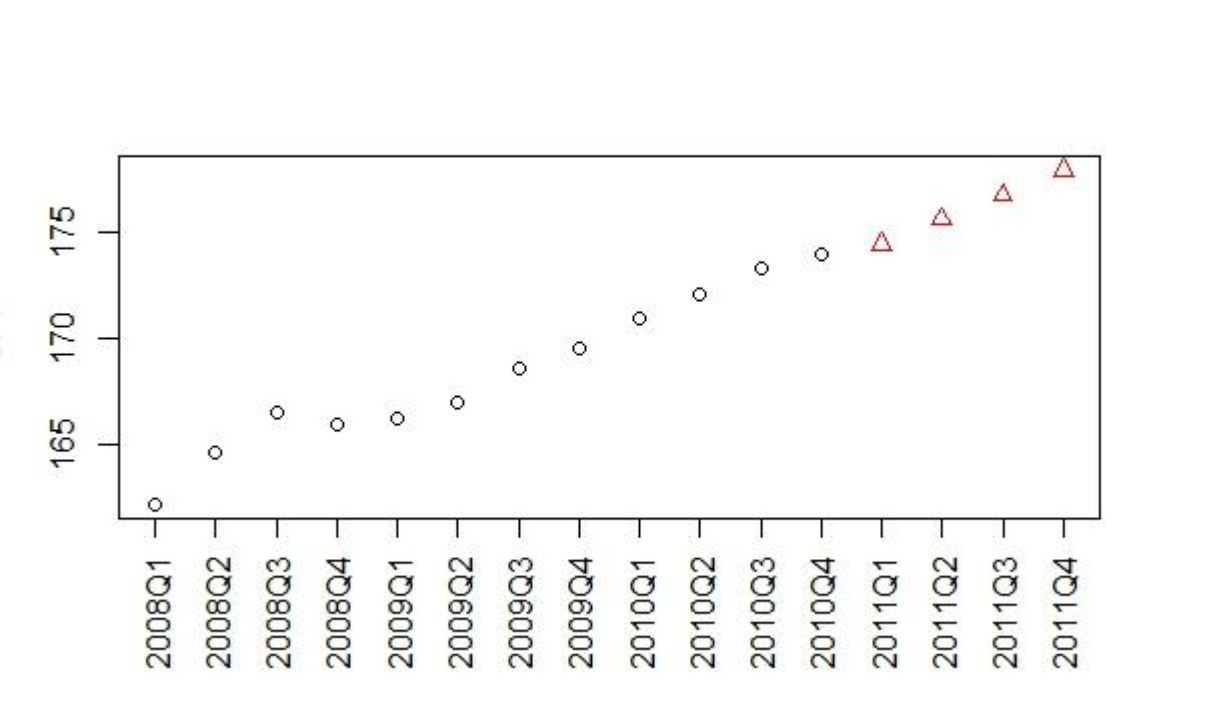
```
data2011 <- data.frame(year = 2011, quarter = 1:4)

cpi2011 <- predict(fit, newdata = data2011)

style <- c(rep(1, 12), rep(2, 4))

plot(c(cpi, cpi2011), xaxt = "n", ylab = "CPI", xlab = "", pch = style, col = style)

axis(1, at = 1:16, las = 3, labels = c(paste(year, quarter, sep = "Q"), "2011Q1",
"2011Q2", "2011Q3", "2011Q4"))
```



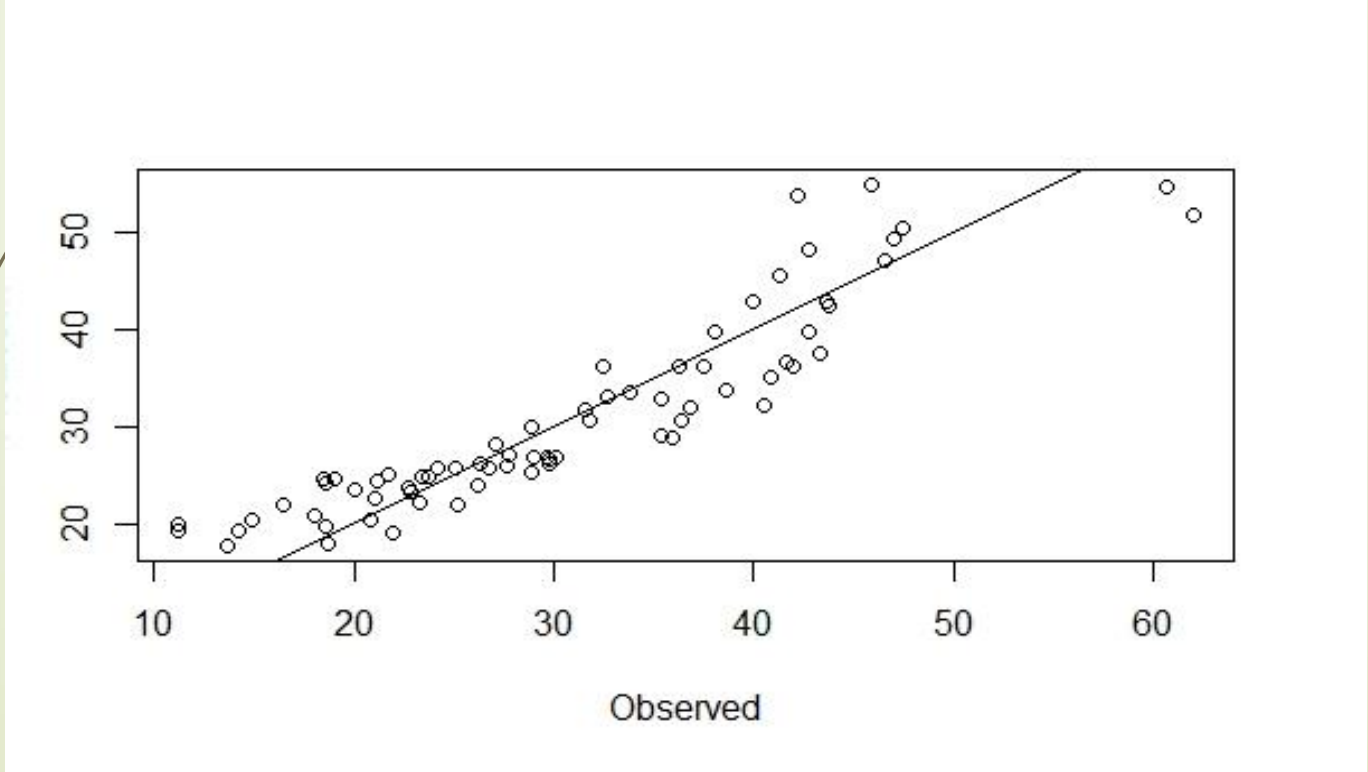
Build a Generalized Linear Model

```
data("bodyfat", package="TH.data")
myFormula <- DEXfat ~ age + waistcirc + hipcirc + elbowbreadth + kneebreadth
bodyfat.glm <- glm(myFormula, family = gaussian("log"), data = bodyfat)
summary(bodyfat.glm)
```

```
## Call:
## glm(formula = myFormula, family = gaussian("log"), data = b...
## Deviance Residuals:
## Min 1Q Median 3Q Max
## -11.569 -3.006 0.127 2.831 10.097
## Coefficients:
## Estimate Std. Error t value Pr(>|t|)
## (Intercept) 0.73429 0.30895 2.38 0.0204 *
## age 0.00213 0.00145 1.47 0.1456
## waistcirc 0.01049 0.00248 4.23 7.4e-05 ***
## hipcirc 0.00970 0.00323 3.00 0.0038 **
## elbowbreadth 0.00235 0.04569 0.05 0.9590
## kneebreadth 0.06319 0.02819 2.24 0.0284
```

Prediction with Generalized Linear Regression Model

```
pred <- predict(bodyfat.glm, type = "response")
plot(bodyfat$DEXfat, pred, xlab = "Observed", ylab = "Prediction")
abline(a = 0, b = 1)
```



R Tutorials

- Introduction to R by Vincent Zoonekynd
- A Handbook of Statistical Analyses Using R by Brian S. Everitt and Torsten Hothorn
- Using R for Data Analysis and Graphics by J H Maindonald
- R Coockbook by O'Reilly
- http://www.cookbook-r.com/
- R Concepts and Data Types presentation by Deepayan Sarkar
- An Introduction to R by D.M. Smith
- Time series Analysis with application R by Kung-Sik Chan
- Analysis of Financial Time Series by Ruey S. Tsay

Thank You